

Lab 2: The Cluster and SLURM

Getting on asacompute, asking the scheduler for exactly what you need, and reading what it gives you back.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

PREREQUISITE

Lab 1, Unit 0

SUBMIT

lab02-<ROLLNO>.pdf

MARKS

10

OFFERING

Odd Semester 2026-27

Why this lab exists

The printed syllabus does not mention job schedulers. It is also impossible to use a shared cluster without one, so this session fills that gap.

Every machine you have programmed so far was yours alone. `asaicompute` is shared by the whole School. If everyone simply ran their programs whenever they liked, sixty jobs would fight over 256 cores and everybody's timings would be noise. A **batch scheduler** exists to stop that: you describe what you need, it decides when and where you run, and while you run those resources are yours.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

- Log in, move files, and find your way around the cluster.
- Read `sinfo` and `squeue` and say what the machine is currently doing.
- Ask for CPUs, memory, GPUs and time, and explain each request.
- Run work interactively with `srun` and in batch with `sbatch`.
- Submit an array job and collect its results.
- Explain why running compute on the login node is antisocial and what to do instead.

Part 0: Getting on

```
ssh <your-username>@asai-11-admin
```

You land on `asaicomputemaster`, the head node. This is a *login* node: you edit files here, compile here, and submit jobs from here.

PITFALL · THE HEAD NODE IS NOT WHERE YOU RUN THINGS

It is shared by everyone who is logged in. A single `./myprogram` that pins 64 cores for ten minutes makes the machine unusable for every other student trying to compile. The scheduler is right there and it is not hard to use.

Compiling: fine. Editing: fine. A two-second test: fine. Anything that takes real time or real memory: `srun` or `sbatch`.

Your home directory is on **GlusterFS**, mounted at `/dist_home/<user>`, and it is the same on every node. A file you write on the head node is visible to your job on node02 without copying anything.

Part 1: What is the machine doing right now

```
sinfo                                     # partitions and node states
sinfo -o "%20N %10c %10m %25G %10T"    # nodes, cpus, memory, GRES, state
squeue                                  # everything queued or running
squeue -u $USER                         # just yours
scontrol show node asaicomputenode02    # everything about one node
```

Exercise 1.1. Run the `sinfo` line above and record the output. Answer, in your report:

- How many nodes are there, and how many CPUs does each have?
- Which nodes have GPUs, how many, and of what type?
- What does the `mixed` state mean, as opposed to `idle` or `alloc`?

Exercise 1.2. Run `squeue`. How many jobs are queued? How many are running? Pick one running job and say how many nodes it holds and for how long it has been running.

NOTE · THE GRES COLUMN

`gpu:a100:2` means that node offers two A100 GPUs as a **generic resource**. You will not be given one unless you ask for it with `--gres=gpu:1`, even if the node you land on has idle GPUs.

Part 2: Interactive work with srun

`srun` runs a command *now*, on a compute node, and shows you the output.

```
srun --cpus-per-task=4 --mem=4G --time=00:05:00 hostname
srun --cpus-per-task=4 --mem=4G --time=00:05:00 nproc
srun --gres=gpu:1 --mem=8G --time=00:05:00 nvidia-smi
```

Exercise 2.1. Run all three. Which node did each land on? Did `nproc` report the node's full core count or your allocation?

DEFINITION · THE FOUR REQUESTS THAT MATTER

`--cpus-per-task` how many cores this task may use. Set `OMP_NUM_THREADS` to match. `--mem` memory for the job; exceed it and the job is killed. `--time` wall-clock limit; exceed it and the job is killed. `--gres=gpu:N` how many GPUs.

Ask for what you need. Asking for more makes you wait longer in the queue and blocks other people. Asking for less gets your job killed halfway through.

Exercise 2.2. Deliberately get a job killed. Run something that asks for `--time=00:00:10` but takes longer:

```
srun --time=00:00:10 --mem=1G sleep 60
```

Record the message. This is what a timeout looks like, and recognising it will save you an hour later in the semester.

Part 3: Batch jobs with sbatch

Real work goes in a script. Create `hello.sbatch` :

```
#!/bin/bash
#SBATCH --job-name=hello
#SBATCH --output=hello-%j.out      # %j is the job id
#SBATCH --error=hello-%j.err
#SBATCH --cpus-per-task=8
#SBATCH --mem=8G
#SBATCH --time=00:05:00

echo "job $SLURM_JOB_ID on $(hostname)"
echo "cores allocated: $SLURM_CPUS_PER_TASK"

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PROC_BIND=close
export OMP_PLACES=cores

./myprogram
```

```
sbatch hello.sbatch      # submit; prints a job id
squeue -u $USER          # watch it
scancel <jobid>          # change your mind
sacct -j <jobid> --format=JobID,State,Elapsed,MaxRSS,ReqCPUS
```

DEFINITION · ALWAYS DERIVE OMP_NUM_THREADS FROM THE ALLOCATION

`export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK` rather than a hard-coded number. Hard-code 96 into a script that gets 8 cores and you will have 96 threads time-slicing across 8 cores, which is slower than 8 threads and produces timings that mean nothing.

Exercise 3.1. Take the OpenMP program from Lab 1 (or any loop you can time) and submit it as a batch job with 8 cores. Confirm from the output file that it ran on a compute node and used 8 threads.

Exercise 3.2. Submit the same program twice, once with `OMP_PROC_BIND=close` and once without. Run each five times. Report the minimum, maximum and spread of the runtimes for both. Which is more reproducible, and why?

Part 4: Array jobs

A **job array** submits the same script many times with a different index. This is how you run a scaling study, a parameter sweep, or the same program at ten different thread counts, without submitting ten jobs by hand.

```
#!/bin/bash
#SBATCH --job-name=scaling
#SBATCH --output=scaling-%A_%a.out    # %A array id, %a task index
#SBATCH --array=0-5
#SBATCH --cpus-per-task=32
#SBATCH --mem=16G
#SBATCH --time=00:10:00

THREADS=(1 2 4 8 16 32)
export OMP_NUM_THREADS=${THREADS[$SLURM_ARRAY_TASK_ID]}
export OMP_PROC_BIND=close
export OMP_PLACES=cores

echo "threads=$OMP_NUM_THREADS"
./myprogram 4096
```

Note `--cpus-per-task=32` for every task, including the one-thread run. That is deliberate: each measurement gets the same physical allocation, so the comparison is fair. Asking for exactly `$OMP_NUM_THREADS` cores would let the one-thread run share a node with somebody else's job and contaminate your baseline.

Exercise 4.1. Run the array above on a program of your choice. Collect the six timings and plot speedup against thread count, with the ideal line.

Exercise 4.2. Compute efficiency and the Karp-Flatt metric e at each thread count. Is e roughly constant, or does it rise? Say what that tells you.

`%A_%a` in the output name matters: without it every array task writes to the same file and you lose five of your six results.

Part 5: Reading what you were given

```
sacct -j <jobid> --format=JobID,JobName,State,Elapsed,TotalCPU,MaxRSS,ReqCPUS,NodeList
seff <jobid> # a readable efficiency summary, if installed
scontrol show job <jobid> # everything, while it is still queued or running
```

Exercise 5.1. For your array job, report `Elapsed`, `TotalCPU` and `MaxRSS` for the 1-thread and 32-thread tasks. Explain the relationship between `Elapsed` and `TotalCPU` in each case. (This is the wall-clock versus CPU-time distinction from Unit 0, appearing in a real tool.)

Exercise 5.2. Look at `MaxRSS`. Did you ask for a sensible `--mem`? If you asked for 16 GB and used 400 MB, you were making other people wait for memory you never touched. What would you request next time?

Part 6: Cluster etiquette

Not marked, and more important than most things that are.

- **Never compute on the login node.** Everything else follows from this one.
- **Always set `--time`.** A job with no limit takes the partition default, which may be hours, and holds resources long after it has hung.
- **Ask for what you need.** Both over-asking and under-asking cause problems.
- **Clean up.** `/dist_home` is shared. Delete the 40 GB of intermediate files when you are done.
- **Check before you complain.** `squeue` will usually show you that the machine is simply busy.
- **`scancel` your mistakes.** A forgotten job holding 64 cores for six hours is noticed.

What to submit

A single PDF, `lab02-<ROLLNO>.pdf`, containing:

1. Your answers to Exercises 1.1 through 5.2.
2. The two batch scripts you wrote, as text (not screenshots).
3. Your speedup and efficiency plot from Part 4, with the ideal line, and the axes labelled.
4. A short paragraph on Exercise 3.2: did thread pinning change the spread of your timings, and by how much?

MARKS	FOR
2	Parts 1 and 2, correctly read and reported
2	A working batch script with the allocation derived from SLURM variables
3	The array job, with a correct speedup and efficiency plot
2	Karp-Flatt interpretation and the <code>sacct</code> reading in Part 5
1	Report is reproducible: machine, compiler, flags, sizes, repetitions stated

NOTE · SCREENSHOTS OF A TERMINAL ARE NOT EVIDENCE

Paste text. It is searchable, it is readable at any size, and it can be checked. This applies to every report in this course.